

SOFTWARE REQUIREMENTS SPECIFICATION

NeoBank Platform — Sprint 5 — Advanced Analytics, Enhancements, Deployment & Presentation

Project	NeoBank Platform
Document	Software Requirements Specification (SRS)
Sprint	Sprint 5 — Advanced Analytics, Enhancements, Deployment & Presentation
Duration	20 Working Days (Days 41–60 of 60-Day Lifecycle)
Program	PMIS (Pradhan Mantri Internship Scheme) — Infosys Limited
Location	Bhubaneswar Development Centre — Lab 4 & Lab 5
Version	1.0 — Draft, Internal Review
Date	April–May 2026
Status	Active — Final Sprint In Progress
Previous Sprint	Sprint 4 — Insights & Admin Controls (Days 31–40)
Next Milestone	Project Sign-Off — NeoBank 360 Formal Handover

© Infosys Limited · Confidential · Not for External Distribution

1. INTRODUCTION

The NeoBank Platform is an enterprise-grade digital banking solution developed under the PMIS (Pradhan Mantri Internship Scheme) Internship Program at the Infosys Bhubaneswar Development Centre, Lab 4 and Lab 5. This document is the authoritative Software Requirements Specification (SRS) for Sprint 5, covering the final 20 working days (Days 41 through 60) of the 60-day development lifecycle.

With all foundational retail banking features — Accounts, Budgets, Bills, Loans, and the Insights & Admin Controls layer — fully operational across Sprints 1 through 4, this final sprint elevates the NeoBank platform to an enterprise standard and prepares it for formal handover. Sprint 5 encompasses four structured phases: advanced graphical analytics for both administrators and end-users; comprehensive system enhancements covering performance, UX polish, and testing; production deployment to cloud infrastructure; and formal documentation, slide deck creation, and project presentation preparation for intern evaluation and sign-off.

1.1 Purpose

This SRS defines the complete set of functional and non-functional requirements, system behaviour, business rules, integration strategy, quality assurance plan, deployment procedures, and sprint-level execution roadmap for NeoBank Sprint 5. It serves as the binding technical reference for all interns, mentors, and stakeholders in Lab 4 and Lab 5 through to project close.

1.2 Scope of Sprint 5

Sprint 5 spans 20 working days across four phases. Deliverables include:

- FR-7: Advanced Admin Analytics — real-time graphical monitoring of Loans, Accounts, Transactions, Bills, Budgets, and a System Audit Log dashboard
- FR-8: Advanced User Analytics — interactive spending habit visualisations, budget adherence charts, loan amortization trajectories, and net worth progression
- NFR-1: Application Enhancements — N+1 query elimination, composite database indexing, Spring Cache implementation, Angular UX polish, global error handling, and skeleton loaders
- NFR-2: System Testing & Deployment — comprehensive unit, integration, E2E, and load testing; Docker containerisation; and cloud deployment to production
- NFR-3: Documentation & Presentation — finalised OpenAPI specs, system architecture documents, user manuals, formal slide decks, live demo scripts, dry runs, and formal project sign-off

1.3 Project Context

Internship Program	PMIS (Pradhan Mantri Internship Scheme)
Organisation	Infosys Limited
Development Centre	Bhubaneswar DC
Labs	Lab 4 & Lab 5
Project	NeoBank Platform
Sprint Duration	20 Working Days (Days 41–60) across 4 Phases
Total Cycle	60 Working Days — Final Sprint
Previous Sprint	Sprint 4 — Insights & Admin Controls (Days 31–40)

Next Milestone

Project Sign-Off — Formal Handover & Intern Evaluation

1.4 Definitions & Abbreviations

TERM	DEFINITION
SRS	Software Requirements Specification — this document
FR	Functional Requirement — a categorised set of deliverable features
NFR	Non-Functional Requirement — quality, performance, and operational deliverables
JWT	JSON Web Token — signed stateless token used for API authentication
NPA	Non-Performing Asset — a loan with overdue repayments, flagged for admin monitoring
AOP	Aspect-Oriented Programming — Spring AOP used for system audit logging interceptors
SAD	System Architecture Document — high-level technical overview including ER and component diagrams
YTD	Year-To-Date — analytics timeframe from 1 January of the current year to the present day
E2E	End-to-End test — full user journey test from UI through API to database and back
PMIS	Pradhan Mantri Internship Scheme — government internship program hosted by Infosys
DC	Development Centre — Infosys Bhubaneswar facility

2. FUNCTIONAL & NON-FUNCTIONAL REQUIREMENTS OVERVIEW

Sprint 5 delivers two new Functional Requirement modules and three Non-Functional Requirement tracks. All deliverables build upon the full banking platform established in Sprints 1–4 and must be fully complete before the formal project sign-off on Day 60.

2.1 FR-7: Advanced Admin Analytics

This module delivers high-level, real-time graphical monitoring capabilities to the administrator portal, providing actionable operational intelligence across all platform modules.

2.1.1 Transaction & Loan Analytics

- ▶ Backend APIs aggregate daily transaction volumes, average ticket sizes, and NPA ratios for loans
- ▶ Line charts visualise inflow vs. outflow velocity over selectable timeframes: 7 days, 30 days, and Year-To-Date
- ▶ Bar charts show loan distribution by status (Pending, Approved, Rejected) with drill-down by product type
- ▶ All admin analytics endpoints protected with `@PreAuthorize("hasRole('ADMIN')")` — CUSTOMER access returns HTTP 403

2.1.2 System Audit Log Dashboard

- ▶ A Spring AOP interceptor automatically captures API execution times and all 4xx/5xx HTTP error events into a `SystemAuditLog` table
- ▶ Each audit log entry contains: endpoint, HTTP method, response status, execution time in milliseconds, `actingUserId`, and timestamp
- ▶ Sensitive data — passwords, plain-text JWTs, and PII — must never be written to the audit log
- ▶ Angular Admin UI displays the System Logs in a searchable, filterable data grid with graphical health indicators (error rate chart, response time trend)
- ▶ System logs are read-only — no modification or deletion endpoints exist for `SystemAuditLog` records

2.2 FR-8: Advanced User Analytics

This module empowers end-users with rich, interactive visualisations of their personal financial health, giving them data-driven tools for financial planning.

2.2.1 Spending & Budget Analytics

- ▶ Backend endpoints return detailed month-over-month category spending vectors for the authenticated user
- ▶ Doughnut charts visualise expense distribution across budget categories (Groceries, Utilities, Rent, etc.)
- ▶ Stacked bar charts compare set Budget limits versus Actual Expenditure interactively per category
- ▶ All user analytics endpoints enforce JWT `userId` cross-validation — cross-user access returns HTTP 403

2.2.2 Wealth & Liabilities Tracking

- ▶ APIs forecast loan payoff dates based on the current repayment schedule and outstanding principal
- ▶ Reward point accrual history is visualised as a line chart over time

- ▶ Interactive area charts display net worth progression: Total Account Balances minus Outstanding Loan Principal, month by month
- ▶ All wealth data is derived from existing schema tables — no new DDL required

2.3 NFR-1: Application Enhancements

- ▶ N+1 query elimination: @EntityGraph or JOIN FETCH applied to all Hibernate queries exhibiting lazy-loading issues identified via SQL analysis
- ▶ Composite B-Tree indexes added to MySQL on (account_id, transaction_date) and other high-frequency filter combinations
- ▶ @Cacheable annotation applied to static or slow-changing endpoints (e.g., GET /api/loans/products) to reduce database load
- ▶ Angular UX polish: all modules audited on mobile viewports; broken flexbox layouts corrected; smooth CSS transitions added for route changes and chart rendering
- ▶ Global Angular error handling via Snackbar notifications; skeleton loaders implemented for all API-bound views

2.4 NFR-2: System Testing & Deployment

- ▶ Comprehensive JUnit/Mockito unit and integration tests for all FR-7 and FR-8 analytics aggregations
- ▶ Angular Jasmine/Karma tests for all new charting components and caching mechanism verification
- ▶ Full E2E lifecycle test: Register → Login → Fund Account → Create Budget → Execute Transactions → View Analytics
- ▶ Load testing with Postman Runner or JMeter: simulate concurrent users on authentication and transaction endpoints
- ▶ Docker containerisation: Dockerfile for Spring Boot backend; nginx.conf for Angular frontend
- ▶ Cloud deployment: live MySQL database, backend compute instances (e.g., AWS EC2 / Elastic Beanstalk), Angular frontend (Vercel or S3)

2.5 NFR-3: Documentation & Presentation

- ▶ OpenAPI/Swagger documentation finalised and verified against all production endpoints from Day 1 through Day 50
- ▶ System Architecture Document (SAD): ER diagrams, component interaction diagrams, tech stack overview
- ▶ End-User Manual and Admin Operating Guide
- ▶ Formal presentation slide deck: Problem Statement, Solution, Tech Stack, Key Features, Architecture, Analytics, Timeline, Intern Contributions
- ▶ Live demo script with step-by-step narrative and assigned speaking roles for all intern team members
- ▶ Dry runs, technical Q&A mock sessions, and formal project sign-off on Day 60

3. BUSINESS & VALIDATION RULES

The following rules are hard constraints enforced across all Sprint 5 deliverables. Any violation is classified as a defect requiring resolution before project sign-off.

FR-7 & FR-8 — ANALYTICS RULES

REF	CONSTRAINT
BR-01	Admin and User analytics endpoints must utilise optimised aggregation queries with appropriate indexes to prevent database locking during high-traffic periods. Unindexed full-table scans on the transactions table are classified as defects.
BR-02	System audit logs must capture API response times and HTTP error codes only. Sensitive user data — passwords, plain-text JWT strings, PII — must never appear in any log entry. Violations are critical security defects.
BR-03	All FR-7 admin analytics endpoints must be protected with <code>@PreAuthorize("hasRole('ADMIN')")</code> . CUSTOMER or unauthenticated access returns HTTP 403 or 401 respectively.
BR-04	All FR-8 user analytics endpoints must validate JWT userId against the requested resource. Cross-user access returns HTTP 403 Forbidden before any aggregation query executes.
BR-05	Analytics timeframe parameters (7d, 30d, YTD) must be validated server-side. Unrecognised or malformed timeframe values return HTTP 400 Bad Request.
BR-06	SystemAuditLog records are immutable once written. No PUT or DELETE endpoints shall exist for the SystemAuditLog resource. Any such request returns HTTP 405 Method Not Allowed.

NFR — DOCUMENTATION & PRESENTATION RULES

REF	CONSTRAINT
BR-01	API documentation must accurately reflect all final production endpoints. Any discrepancy between the Swagger spec and the live API behaviour is classified as a documentation defect.
BR-02	The final presentation demo must utilise a fully seeded, realistic database state. Sparse or unrepresentative data during the live demo is unacceptable and must be prevented through Day 56 seeding activities.
BR-03	All hardcoded local URLs, credentials, and environment-specific values must be removed from production configuration files before deployment. Their presence is a critical deployment defect.
BR-04	Docker images must be fully self-contained. The application must start with only the required environment variable injections — no manual file edits post-deployment are permitted.
BR-05	Every intern team member must have a clearly assigned and practised segment in the final presentation. Unassigned or unrehearsed segments are flagged during dry runs and corrected before Day 60.

4. INFRASTRUCTURE & ARCHITECTURE

Sprint 5 extends the established three-tier architecture with advanced charting capabilities, Spring AOP for audit logging, Spring Cache for performance, and production-grade deployment infrastructure. New components are additive — no architectural changes are made to existing layers.

4.1 Technology Stack

LAYER	TECHNOLOGY	DETAILS
Frontend	Angular + Chart.js / D3.js	Advanced charting: Doughnut, Stacked Bar, Line, Area charts via ng2-charts or D3.js; Angular Material; global Snackbar error handler; skeleton loaders; responsive CSS audit
Backend	Spring Boot	Spring AOP interceptor for SystemAuditLog; @Cacheable on static endpoints; @EntityGraph / JOIN FETCH for N+1 elimination; @Transactional(readOnly=true) on all analytics reads; JMeter/Postman load targets
Database	MySQL 8.0+	New SystemAuditLog table (Day 41 DDL); composite B-Tree indexes on (account_id, transaction_date); EXPLAIN analysis for aggregation query review
Security	JWT + BCrypt	All FR-7 endpoints: @PreAuthorize("hasRole('ADMIN')"); FR-8 endpoints: JWT userId cross-check; audit log entries never contain JWTs or passwords
Deployment	Docker + AWS / Vercel	Dockerfile for Spring Boot .jar; nginx.conf for Angular dist/; application-prod.yml with environment variables; AWS EC2 / Elastic Beanstalk for backend; Vercel or S3+CloudFront for frontend
Logging	SLF4J + Logback	Structured log output; AOP-captured API metrics written to SystemAuditLog table and log files; log level configurable via application.yml
API Docs	Swagger / OpenAPI	Full documentation of all endpoints from Days 1–50; exported as openapi-final.json; committed to docs/ in both repositories
Testing	JUnit + Jasmine/Karma	JUnit/Mockito for backend analytics; Jasmine/Karma for Angular charting components; Postman Runner / JMeter for load testing

4.2 New Database Table — Day 41

One new table is provisioned on Day 41 via DDL script to support the System Audit Log feature. All other analytics in Sprint 5 are derived from existing schema tables.

4.2.1 system_audit_log

- id — BIGINT, PK, AUTO_INCREMENT
- endpoint — VARCHAR(500), NOT NULL
- http_method — VARCHAR(10), NOT NULL
- response_status — INT, NOT NULL
- execution_time_ms — BIGINT, NOT NULL
- acting_user_id — BIGINT, NULL (NULL for unauthenticated requests)
- event_timestamp — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
- error_message — VARCHAR(1000), NULL (populated only for 4xx/5xx responses)
- INDEX idx_audit_status (response_status) — for filtering by error code
- INDEX idx_audit_timestamp (event_timestamp) — for time-range queries on the log grid

5. INSTALLATION & SETUP — DAY 41

Day 41 is dedicated to Sprint 5 environment extension. Both sessions must be completed before Day 42 feature development begins. One DDL script must be executed and all new libraries verified before the analytics build commences.

5.1 Session 1: Advanced Charting & Frontend Dependencies

Git Branch Setup

- ▶ Create a new Git branch from develop: `git checkout -b feature/sprint5-advanced-analytics-deployment`
- ▶ Push to remote: `git push -u origin feature/sprint5-advanced-analytics-deployment`
- ▶ Share branch access with all team members and the PMIS mentor

Angular Advanced Charting Libraries

- ▶ Verify ng2-charts and Chart.js from Sprint 4 are present; upgrade if needed: `npm install chart.js@latest ng2-charts@latest`
- ▶ For D3.js advanced charts: `npm install d3 @types/d3`
- ▶ Import NgChartsModule and any D3 utility services in AppModule
- ▶ Run `ng serve` to confirm the Angular application starts without errors after all additions

5.2 Session 2: Backend Logging & Database Setup

SLF4J / Logback Configuration

- ▶ Verify SLF4J and Logback are present in pom.xml (included by default with Spring Boot)
- ▶ Configure logback-spring.xml with a RollingFileAppender: max file size 10MB, max history 30 days
- ▶ Set log level for com.neobank to INFO in application.yml; DEBUG in application-dev.yml

DDL Execution

- ▶ `11_create_system_audit_log_table.sql` — creates system_audit_log with indexes on response_status and event_timestamp
- ▶ Verify: `SHOW TABLES`; confirms 11 tables present
- ▶ Verify: `DESCRIBE system_audit_log`; — validate all columns and indexes

Spring AOP & Cache Scaffolding

- ▶ Add spring-boot-starter-aop to pom.xml if not already present
- ▶ Create AuditLogAspect class with `@Aspect` and `@Around("execution(* com.neobank.controller..*(..))")`
- ▶ Add Spring Cache: `@EnableCaching` in main application class; configure CaffeineCacheManager or SimpleCacheManager
- ▶ Run the Spring Boot application to confirm AOP and cache beans initialise without errors

6. SPRINT 5 EXECUTION PLAN

The 20-day final sprint is structured across four phases. Integration testing is woven throughout Phases 1 and 2. Each session is approximately 3–4 hours of focused work.

6.1 Sprint Schedule Overview

DAY	PHASE	SESSION 1	SESSION 2
41	Phase 1: Setup	Create Sprint 5 Git branch; upgrade Angular charting libs; configure D3.js	Execute SystemAuditLog DDL; configure Logback; scaffold AOP interceptor and Spring Cache
42	Phase 1: Analytics	Backend APIs: daily transaction volumes, average ticket size, NPA ratios for loans	Angular Admin graphs: Line/Bar charts for inflow vs. outflow; loan distribution over 7d/30d/YTD
43	Phase 1: Analytics	Spring AOP interceptor: log execution times and 4xx/5xx errors to SystemAuditLog table	Angular Admin System Logs grid: searchable, filterable; error rate and response time health charts
44	Phase 1: Analytics	Backend: month-over-month category spending vectors for authenticated user	Angular User Analytics: Doughnut chart (expense distribution); Stacked Bar (Budget vs. Actual)
45	Phase 1: Analytics	Backend APIs: loan payoff forecast; reward point accrual history time series	Angular: area charts for net worth progression (account balances minus outstanding loan principal)
46	Phase 2: Enhance	Analyse Hibernate SQL; apply @EntityGraph / JOIN FETCH; add composite MySQL indexes	@Cacheable on GET /api/loans/products and other static endpoints; verify cache hits/misses
47	Phase 2: Enhance	Audit Angular modules on mobile viewports; fix flexbox layouts; add route-change CSS transitions	Global Angular Snackbar error handler; skeleton loaders for all API-bound components
48	Phase 2: Testing	JUnit/Mockito: unit tests for FR-7 and FR-8 aggregation accuracy and edge cases	Jasmine/Karma: Angular tests for charting components; cache eviction and @Cacheable verification tests
49	Phase 2: Testing	E2E full lifecycle tests: Register → Login → Fund → Budget → Transact → Analytics	Defect triage and hotfixes for all broken flows identified during E2E execution
50	Phase 2: Testing	Load test with Postman Runner / JMeter on auth and transaction endpoints; concurrent user simulation	Monitor MySQL CPU and Spring Boot heap; tune connection pool and memory limits as needed
51	Phase 3: Docs	Finalise OpenAPI/Swagger: all endpoints Days 1–50; export openapi-final.json to docs/	Complete README.md: setup instructions, environment variables, Docker run commands
52	Phase 3: Docs	Draft SAD: ER diagrams, component interaction diagrams, tech stack overview	Create End-User Manual and Admin Operating Guide for dashboards and loan approval workflows
53	Phase 3: Deploy	Finalise application-prod.yml; remove all hardcoded local URLs; set all env vars	Write Dockerfile for Spring Boot backend; write nginx.conf for Angular dist/ serving
54	Phase 3: Deploy	Provision live MySQL DB on cloud; setup backend compute instance (EC2 / Elastic Beanstalk)	Build final .jar and dist/; deploy to live servers; connect frontend domain to backend API Gateway

DAY	PHASE	SESSION 1	SESSION 2
55	Phase 3: Deploy	Live smoke testing on production URLs: CORS, SSL, DB connectivity verification	Verify Admin and User Analytics dashboards reading live production DB under real network conditions
56	Phase 4: Present	Final regression: re-run critical path tests on live production environment	Finalise demo data seeding: thousands of realistic transactions and loan applications for presentation
57	Phase 4: Present	Draft formal slide deck: Problem Statement, Solution, Tech Stack, Key Features, Architecture	Add analytics visuals, DB schemas, execution timeline slide; intern contribution highlights
58	Phase 4: Present	Write step-by-step live demo script with clear narrative (e.g., 'Meet Arun, a customer...')	Assign speaking roles; every intern gets a dedicated segment; rehearse transitions
59	Phase 4: Present	Full timed dry runs of presentation and live demo; smooth speaker handoffs	Technical Q&A mock sessions; evaluate intern understanding of system design and implementation
60	Phase 4: Sign-Off	Final slide deck polish; production environment pre-presentation checks	Deliver formal presentation and live demo to stakeholders; secure project sign-off

6.2 Phase 1: Advanced Analytics (Days 42–45)

Day 42 — Advanced Admin Analytics: Transactions & Loans

- Backend: GET /api/admin/analytics/transactions?timeframe={7d|30d|YTD} — returns dailyVolumes[], averageTicketSize, totalInflow, totalOutflow
- Backend: GET /api/admin/analytics/loans?timeframe={7d|30d|YTD} — returns loanDistribution by status, NPA count and ratio
- All admin analytics endpoints annotated @PreAuthorize("hasRole('ADMIN')")
- Angular: Line chart — inflow vs. outflow velocity; X-axis = date, Y-axis = amount; timeframe selector (7d / 30d / YTD) refreshes chart
- Angular: Bar chart — loan distribution by status (Pending / Approved / Rejected); grouped by product type

Day 43 — System Audit Log

- AuditLogAspect.@Around intercepts all controller method executions: record endpoint, method, response status, execution_time_ms, acting_user_id
- Error capture: for any ProceedingJoinPoint that throws or returns 4xx/5xx, populate error_message with the sanitised exception message — never the raw stack trace
- AuditLogRepository persists SystemAuditLog entities; all writes via @Async to avoid blocking the main request thread
- GET /api/admin/system-logs?from=&to=&status= — paginated, filterable log fetch — @PreAuthorize ADMIN only
- Angular Admin: searchable data grid with columns: Timestamp, Endpoint, Method, Status, Execution Time (ms), Error
- Angular Admin: error rate mini-chart (% 4xx+5xx per hour) and response time trend line chart above the grid

Day 44 — Advanced User Analytics: Spending & Budgets

- GET /api/analytics/spending/{userId}?months=6 — returns categorySpending[] with month-over-month vectors; JWT userId cross-check

- ▶ Angular: Doughnut chart — expense distribution across categories for the selected month
- ▶ Angular: Stacked Bar chart — Budget limit (set) vs. Actual spend per category; up to 6 months displayed
- ▶ Month selector component allows the user to navigate between months; chart re-renders on selection change

Day 45 — Advanced User Analytics: Wealth & Liabilities

- ▶ GET /api/analytics/wealth/{userId} — returns netWorthTimeline[] (month, totalBalance, outstandingPrincipal, netWorth), loanPayoffForecast[], rewardAccrualHistory[]
- ▶ netWorth = SUM(account balances) – SUM(outstanding loan principal) per month
- ▶ loanPayoffForecast: for each active loan account, calculate months remaining and projected payoff date from current repayment schedule
- ▶ Angular: Area chart — net worth progression; shaded area between total balance and outstanding principal lines
- ▶ Angular: Line chart — reward point accrual over time; points on Y-axis, months on X-axis

6.3 Phase 2: Enhancements & Testing (Days 46–50)

Day 46 — Database & Query Enhancement

- ▶ Run EXPLAIN SELECT on all analytics queries; identify full-table scans and missing index usage
- ▶ Apply composite indexes: ALTER TABLE transactions ADD INDEX idx_txn_account_date (account_id, transaction_date)
- ▶ Apply @EntityGraph on LoanApplication repository to fetch user and product in one JOIN instead of N+1 lazy loads
- ▶ Apply @Cacheable(value="loanProducts", key="all") on LoanProductService.findAll(); add @CacheEvict on create/update
- ▶ Verify cache hit: call GET /api/loans/products twice in Postman — second call must not trigger a DB query (verify via Spring Boot log)

Day 47 — UI Enhancements

- ▶ Mobile viewport audit: test all Angular routes at 375px width; fix overflow, truncation, and hidden button issues
- ▶ Add route-change animation: import RouterModule.forRoot with RouteReuseStrategy and @angular/animations BrowserModule
- ▶ Implement global HttpInterceptor for error responses: display Angular MatSnackBar with friendly error message for 4xx/5xx
- ▶ Add skeleton loaders (shimmer CSS) to: dashboard, insights, loan repayment schedule, and admin analytics pages

Day 48 — Unit & Integration Testing

- ▶ JUnit: verify totalInflow + totalOutflow = sum of all active-account transactions in test timeframe
- ▶ JUnit: verify NPA ratio = count(OVERDUE loan accounts) / count(all loan accounts) × 100
- ▶ JUnit: verify AOP interceptor writes SystemAuditLog entry for each controller call in test context
- ▶ JUnit: verify @Cacheable prevents a second DB call; verify @CacheEvict clears the cache on product creation
- ▶ Jasmine: verify Doughnut chart renders correct number of segments matching category count from API response
- ▶ Jasmine: verify Area chart data arrays have correct length matching months returned

Day 49 — E2E Testing & Bug Fixing

- ▶ E2E lifecycle: Register new user → Login → Create SAVINGS account → Credit 50,000 → Create Groceries budget → Debit 5,000 → View insights → View spending analytics → Verify Doughnut chart has Groceries segment
- ▶ Admin E2E: Login as admin → Apply loan as customer → Approve from admin dashboard → Verify loan appears in analytics loan distribution chart
- ▶ Log all discovered defects with severity: Critical / High / Medium / Low
- ▶ Execute hotfixes for Critical and High severity defects before Day 50

Day 50 — Performance & Load Testing

- ▶ Postman Runner: 50 concurrent users running POST /api/auth/login; target p95 response time < 300ms
- ▶ Postman Runner: 30 concurrent users running POST /api/accounts/{id}/transactions; target p95 < 500ms
- ▶ JMeter (optional): ramp-up 100 users over 60 seconds on the insights endpoint; monitor Spring Boot heap in VisualVM
- ▶ MySQL monitoring: run SHOW PROCESSLIST during load tests; identify any blocking queries
- ▶ Tune HikariCP connection pool size in application.yml if connection timeouts observed

6.4 Phase 3: Documentation & Deployment (Days 51–55)

Day 51 — API & Technical Documentation

- ▶ Open Swagger UI; verify every endpoint from Days 1–50 is documented with correct request/response schemas
- ▶ Update @Operation, @ApiResponse, @Schema annotations for any FR-7 and FR-8 endpoints added in Sprint 5
- ▶ Export: GET http://localhost:8080/v3/api-docs → save as docs/openapi-final.json and commit
- ▶ Update README.md: add Docker build commands, environment variable reference table, and deployment guide section

Day 52 — System Architecture & User Manuals

- ▶ Draft SAD in docs/system-architecture.md: include MySQL ER diagram (all 11 tables), Spring Boot component diagram, Angular module tree
- ▶ Document the tech stack decision rationale: why Angular over React, why MySQL over NoSQL, JWT over sessions
- ▶ End-User Manual (docs/user-manual.md): how to register, manage accounts, set budgets, pay bills, apply for loans, and interpret analytics charts
- ▶ Admin Operating Guide (docs/admin-guide.md): how to approve loans, manage users, read system logs, and interpret admin dashboards

Day 53 — Pre-Deployment Configuration

- ▶ Create application-prod.yml: replace all hardcoded values with environment variable placeholders: \${DB_URL}, \${DB_USER}, \${DB_PASS}, \${JWT_SECRET}
- ▶ Set CORS allowed origins to the production frontend domain only — remove localhost from production config
- ▶ Write Dockerfile: FROM eclipse-temurin:17-jdk-alpine; COPY target/*.jar app.jar; ENTRYPOINT ["java", "-jar", "app.jar"]
- ▶ Write nginx.conf: serve Angular dist/neobank/ at /; proxy /api/* to backend container on port 8080
- ▶ Test Docker build locally: docker build -t neobank-backend . && docker run -p 8080:8080 neobank-backend

Day 54 — Infrastructure Provisioning & Deployment

- ▶ Cloud MySQL: provision RDS MySQL 8.0 instance (or equivalent); run all 11 DDL scripts; run seed scripts for demo data
- ▶ Backend deployment: push Docker image to ECR (or equivalent); deploy to EC2 / Elastic Beanstalk with env vars injected
- ▶ Frontend deployment: run ng build --configuration production; deploy dist/ to Vercel or S3+CloudFront
- ▶ Configure API Gateway or load balancer to route /api/* to backend; serve Angular at the root domain

Day 55 — Post-Deployment Smoke Testing

- ▶ Test POST /api/auth/login on production URL; verify JWT returned and stored correctly by Angular
- ▶ Test GET /api/admin/dashboard from production frontend; verify CORS headers and response payload
- ▶ Verify SSL certificate: HTTPS only; HTTP requests redirected to HTTPS
- ▶ Verify Angular routing: hard-refresh on /insights and /admin/dashboard must not return 404 (nginx try_files config)
- ▶ Verify Admin and User Analytics charts load correctly from the live production database under real network latency

6.5 Phase 4: Presentation & Handover Prep (Days 56–60)

Day 56 — Final Regression & Demo Seeding

- ▶ Re-run all critical path Postman tests against the production URL; confirm all pass
- ▶ Seed demo database: insert 5,000+ realistic transactions spread over 12 months for demo user accounts
- ▶ Seed 20+ loan applications across all statuses; seed 6 months of budget data for all categories
- ▶ Verify all analytics charts look visually rich and populated during a pre-presentation walkthrough

Day 57 — Slide Deck Creation

- ▶ Slide 1: Title — NeoBank 360, team name, date, Infosys PMIS program
- ▶ Slides 2–3: Problem Statement and Solution Overview
- ▶ Slides 4–5: Tech Stack and System Architecture diagram from SAD
- ▶ Slides 6–8: Key Features walkthrough — Core Banking, Budgeting, Loans, Analytics
- ▶ Slides 9–10: Advanced Analytics screenshots — Admin and User dashboards
- ▶ Slide 11: Execution Timeline — 60-day sprint overview with milestones
- ▶ Slide 12: Intern Contributions — individual module ownership and achievements
- ▶ Slide 13: Challenges & Learnings
- ▶ Slide 14: Live Demo Notice — transition to screen share

Day 58 — Live Demo Script

- ▶ Narrative: 'Meet Arun, a NeoBank customer who wants to manage his finances and apply for a home loan...'
- ▶ Demo flow: Register Arun → Login → Create Savings Account → Credit salary → Set budgets → Pay bills → Apply for Home Loan → Switch to Admin → Approve loan → Return to Arun → View repayment schedule → View analytics charts
- ▶ Each intern assigned one specific demo segment; transitions scripted explicitly
- ▶ Fallback screenshots prepared for each step in case of live environment issues

Day 59 — Dry Runs & Q&A Preparation

- ▶ Full timed dry run: target total presentation + demo duration 20–25 minutes

- ▶ Check slide transitions, screen share handoffs, and microphone quality
- ▶ Technical Q&A mock: questions drawn from the entire 60-day SRS set — architecture decisions, API design, security model, EMI formula, analytics aggregations
- ▶ Each intern answers at least 2 technical questions in the mock session

Day 60 — Final Presentation & Project Sign-Off

- ▶ Pre-presentation: final check of production environment, slide deck, and demo account data
- ▶ Session 1: deliver formal slide deck presentation to stakeholders and PMIS mentor
- ▶ Session 2: deliver live application demo following the Day 58 script
- ▶ Post-demo: formal Q&A session; PMIS mentor evaluates each intern against project objectives
- ▶ Secure official project sign-off, formally concluding the 60-day NeoBank 360 development lifecycle

7. API REFERENCE

All secured endpoints require a valid JWT Bearer token in the Authorization header. Endpoints inherited from Sprints 1–4 remain unchanged and are not repeated here.

FR-7 — ADVANCED ADMIN ANALYTICS ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
GET /api/admin/analytics/transactions	GET	Day 42	ADMIN	Daily transaction volumes, average ticket size, total inflow/outflow. Supports ?timeframe= query param (7d/30d/YTD).
GET /api/admin/analytics/loans	GET	Day 42	ADMIN	Loan distribution by status; NPA count and ratio. Supports ?timeframe= query param.
GET /api/admin/system-logs	GET	Day 43	ADMIN	Paginated SystemAuditLog fetch. Supports ?from=&to=&status= filters. Ordered by timestamp descending.

FR-8 — ADVANCED USER ANALYTICS ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
GET /api/analytics/spending/{userId}	GET	Day 44	JWT	Month-over-month category spending vectors. Supports ?months= param (default 6). 403 if userId mismatches JWT.
GET /api/analytics/wealth/{userId}	GET	Day 45	JWT	Net worth timeline, loan payoff forecasts, and reward accrual history. 403 if userId mismatches JWT.

8. INTEGRATION STRATEGY

Integration for Sprint 5 is continuous — each analytics feature is wired end-to-end within the same day it is built rather than deferred to a dedicated integration session. Full regression integration is conducted on Day 49 (E2E testing) and Day 55 (production smoke testing).

8.1 Daily Analytics Integration (Days 42–45)

- ▶ Each day's Session 2 wires the Angular component built that day to the Session 1 backend endpoint
- ▶ Verify JWT is correctly attached by `HttpInterceptor` on every analytics API call
- ▶ Verify chart datasets are correctly mapped from API JSON response to `Chart.js` dataset format
- ▶ Verify HTTP 403 is displayed correctly in Angular when a CUSTOMER user attempts an admin analytics endpoint
- ▶ Verify chart gracefully handles empty datasets: no errors thrown when a new user with no transactions views analytics

8.2 Day 49 — Full E2E Integration

- ▶ Full user lifecycle executed in a clean test database: Register → Login → Fund Account → Create Budget → Execute Transactions → View Insights → View Spending Analytics → View Wealth Chart
- ▶ Admin lifecycle: Login as Admin → Review pending loans → Approve loan → Verify loan in analytics distribution chart → View system audit log for approval API call entry
- ▶ Verify AOP audit log captures the loan approval API call with correct endpoint, status 200, and execution time
- ▶ Verify `@Cacheable`: call `GET /api/loans/products` twice; confirm Spring Boot log shows cache hit on second call

8.3 Day 55 — Production Integration Verification

- ▶ All integration tests from Day 49 re-executed against the live production URL
- ▶ Verify CORS: Angular production domain included in Spring Boot allowed origins; localhost removed
- ▶ Verify environment variables: confirm no hardcoded values leak into production logs or API responses
- ▶ Verify analytics charts load with live data under real network latency — target initial chart render under 2 seconds

9. DATA SEEDING

Sprint 5 requires two rounds of data seeding: an initial technical seed for analytics testing (Day 41–42) and a rich presentation-quality seed for the Day 56 demo environment. All prior sprint seed data is preserved.

ENTITY	COUNT	SEED DETAILS
Transaction History	5,000+	Realistic CREDIT and DEBIT transactions spread across 12 months for demo user accounts; varying amounts across all budget categories to produce rich chart distributions
Loan Applications	20+	Mix of PENDING, APPROVED, and REJECTED across all 3 configured loan products; at least 3 OVERDUE repayments to demonstrate NPA ratio on admin analytics
Budget Records	18+	6 months of budget data across all 6 categories for demo users; varying limits and expenditures to produce meaningful Budget vs. Actual stacked bar charts
System Audit Log	100+	Pre-seeded audit log entries including a mix of 200, 400, and 500 status codes to populate the System Log grid and error rate chart for demo
User Accounts	10+	Mix of active and inactive users; admin and customer roles; realistic full names and emails for the demo narrative
Reward Points	N/A	Reward point accrual history seeded for demo users spanning 12 months to generate a meaningful reward accrual trend line chart

10. QUALITY ASSURANCE & TESTING

Testing in Sprint 5 spans Days 48–50 for structured test execution and Day 59 for technical Q&A evaluation of intern competencies. All critical and high-severity defects must be resolved before deployment on Day 54.

DAY 48 — FR-7 & FR-8 ANALYTICS UNIT TESTS

Unit Tests

- ▶ Transaction analytics — totalInflow: seed 10 CREDIT transactions → assert API returns exact sum within selected timeframe
- ▶ Transaction analytics — timeframe filtering: seed transactions outside the 7-day window → assert they do NOT appear in 7d response
- ▶ NPA ratio: seed 2 OVERDUE loan accounts out of 10 total → assert npaRatio = 20.00
- ▶ Category spending: seed 3 categories with known amounts for 3 months → assert month-over-month vectors match
- ▶ Net worth: seed account balance 100,000 and outstanding loan principal 40,000 → assert netWorth = 60,000
- ▶ AOP audit log: execute any controller method in test context → assert exactly one SystemAuditLog entry written
- ▶ AOP sensitive data: verify no entry in SystemAuditLog contains the string 'password' or 'Bearer'
- ▶ Cache test: call findAll() twice → assert underlying repository method invoked exactly once (Mockito verify)

DAY 48 — FRONTEND COMPONENT TESTS

Jasmine / Karma Tests

- ▶ Doughnut chart component: mock API response with 4 categories → assert chart renders 4 data segments
- ▶ Stacked bar chart: mock 3 months of budget vs. actual data → assert 3 bar groups rendered with correct dataset labels
- ▶ Area chart: mock 6-month net worth timeline → assert 6 data points in each dataset
- ▶ Snackbar error handler: mock HTTP 500 from interceptor → assert MatSnackBar.open() called with expected message
- ▶ Skeleton loader: assert loading class present on component init; assert removed after API observable emits

DAY 49 — E2E & BUG FIXING

End-to-End Tests

- ▶ Full lifecycle: Register → Login → Fund Account (50,000) → Debit (Groceries, 5,000) → View Spending Doughnut → Verify Groceries segment present
- ▶ Admin analytics E2E: Approve a loan → confirm loan appears in Approved bar of loan distribution chart
- ▶ System log E2E: perform 5 API calls → navigate to Admin System Logs grid → confirm 5 new entries visible
- ▶ Net worth E2E: credit account 100,000 → approve loan (40,000) → view wealth chart → verify netWorth = 60,000 in chart data
- ▶ CUSTOMER admin route guard: login as customer → manually navigate to /admin/analytics → verify redirect to /dashboard

DAY 50 — PERFORMANCE & LOAD TESTING

Load Test Targets

- ▶ POST /api/auth/login — 50 concurrent users → p95 response time < 300ms; zero 5xx errors
- ▶ POST /api/accounts/{id}/transactions — 30 concurrent users → p95 response time < 500ms; zero 5xx errors
- ▶ GET /api/insights/{userId} — 20 concurrent users → p95 response time < 600ms
- ▶ GET /api/admin/analytics/transactions — 10 concurrent admin users → p95 response time < 800ms
- ▶ Database: SHOW PROCESSLIST during load run; zero blocking queries; connection pool utilisation < 80%

11. DOCUMENTATION & SPRINT WRAP-UP

Sprint 5 concludes the entire NeoBank 360 project. Day 60 is the formal endpoint of the 60-day lifecycle. All deliverables below must be committed to Git and demonstrated live before the project is considered signed off.

11.1 API Documentation (Day 51)

- ▶ All endpoints from Days 1–50 verified in Swagger UI and accurately described
- ▶ FR-7 and FR-8 endpoints annotated with @Operation, @ApiResponse, @Schema including timeframe parameters
- ▶ Exported openapi-final.json committed to docs/ in neobank-backend repository
- ▶ README.md in both repositories updated with complete Docker build, run commands, and environment variable reference

11.2 System Architecture Document (Day 52)

- ▶ docs/system-architecture.md: ER diagram of all 11 tables with FK relationships
- ▶ Component interaction diagram: Angular modules → HTTP Interceptor → Spring Controllers → Services → Repositories → MySQL
- ▶ Tech stack justification section: rationale for each technology choice
- ▶ Deployment architecture diagram: Angular (CDN) → API Gateway → Spring Boot (EC2) → RDS MySQL

11.3 User & Admin Documentation (Day 52)

- ▶ docs/user-manual.md: step-by-step guide for registration, account management, budgeting, bill payments, loan applications, and reading analytics charts
- ▶ docs/admin-guide.md: guide for loan approval workflow, user status management, reading system logs, and interpreting admin analytics dashboards

11.4 Presentation & Demo Deliverables (Days 57–59)

- ▶ Formal slide deck (14 slides minimum) committed to docs/presentation/ as PDF and source format
- ▶ Live demo script committed to docs/demo-script.md with step-by-step narrative, screen share instructions, and assigned speaker names
- ▶ Fallback screenshots committed to docs/demo-screenshots/ for each demo step
- ▶ Question bank committed to docs/qna-bank.md — minimum 20 technical questions covering all 5 sprints

11.5 Final Project Sign-Off (Day 60)

- ▶ Formal live demo delivered to stakeholders and PMIS mentor
- ▶ All 12 acceptance criteria (Section 12) verified as PASSED
- ▶ Git repositories clean: all feature branches merged to main; final tag v1.0.0 created
- ▶ PMIS mentor formal sign-off received, concluding the NeoBank 360 60-day development lifecycle

12. ACCEPTANCE CRITERIA

All criteria below constitute the Definition of Done for Sprint 5 and the entire NeoBank 360 project. Every item must be met before the project can be formally signed off on Day 60.

FR-7 — ADVANCED ADMIN ANALYTICS

#	CRITERION	VERIFIED BY
AC01	Transaction analytics API returns correct inflow, outflow, and daily volumes for 7d, 30d, and YTD timeframes	Unit Test + Postman
AC02	Loan distribution API returns correct counts per status; NPA ratio matches OVERDUE loan count / total loans	Unit Test
AC03	Spring AOP interceptor writes SystemAuditLog entry for every controller call; sensitive data absent from all entries	Unit Test
AC04	All FR-7 endpoints restricted to ADMIN role; CUSTOMER access returns HTTP 403 Forbidden	Unit Test
AC05	Angular Admin analytics charts (Line, Bar) render with correct data and timeframe selector updates chart without reload	E2E Test
AC06	Angular System Logs grid displays audit entries; error rate and response time charts render correctly	E2E Test

FR-8 — ADVANCED USER ANALYTICS

#	CRITERION	VERIFIED BY
AC07	Spending API returns correct category vectors; Doughnut and Stacked Bar charts render matching data	Unit Test + E2E
AC08	Net worth calculation: totalBalance – outstandingPrincipal matches expected value for seeded test data	Unit Test
AC09	Loan payoff forecast returns correct months remaining for each active loan account	Unit Test + Postman
AC10	Cross-user access to FR-8 analytics endpoints rejected with HTTP 403 Forbidden	Unit Test

NFR-1 — ENHANCEMENTS

#	CRITERION	VERIFIED BY
AC11	@Cacheable on GET /api/loans/products: second call does not trigger a DB query (Spring Boot log verified)	Unit Test
AC12	EXPLAIN SELECT on key analytics queries shows index usage; no full-table scans on transactions table	Performance Review
AC13	All Angular modules render correctly at 375px mobile viewport; no overflow or hidden content	Manual Review
AC14	Global Snackbar error handler displays friendly message for all 4xx/5xx API responses	E2E Test

NFR-2 — TESTING & DEPLOYMENT

#	CRITERION	VERIFIED BY
AC15	All unit tests pass with zero critical failures across FR-7, FR-8, and NFR-1	Test Suite
AC16	Load test: p95 response time < 300ms for login endpoint under 50 concurrent users; zero 5xx errors	Load Test
AC17	Application runs successfully in Docker container with environment variables injected; no hardcoded values in prod config	Deployment Review
AC18	Production deployment live: Angular frontend accessible via HTTPS; Spring Boot API responding; MySQL connected	Smoke Test

NFR-3 — DOCUMENTATION & PRESENTATION

#	CRITERION	VERIFIED BY
AC19	Swagger UI reflects all endpoints from all 5 sprints with correct schemas; openapi-final.json committed	Manual Review
AC20	SAD committed to docs/ with ER diagram, component diagram, and deployment architecture	Mentor Review
AC21	Formal slide deck (minimum 14 slides) and live demo script committed and presented on Day 60	Mentor Review
AC22	Every intern demonstrates understanding of their module in technical Q&A mock and final presentation	Mentor Evaluation

GENERAL QUALITY

#	CRITERION	VERIFIED BY
AC23	Git repositories: all Sprint 5 branches merged to main; final tag v1.0.0 created; commit history clean	Mentor Review
AC24	No hardcoded credentials in any file; production config uses environment variables exclusively	Code Review
AC25	Official PMIS mentor sign-off received on Day 60 concluding the 60-day NeoBank 360 lifecycle	Mentor Sign-Off

Prepared under the PMIS (Pradhan Mantri Internship Scheme) · Infosys Bhubaneswar DC · Lab 4 & Lab 5
© Infosys Limited · Confidential · Not for External Distribution